

Sovereign On-Premise Agentic AI Workbench

Team Reference & Presentation Pack

PROBLEM STATEMENT

PS 26117 — *Sovereign On-Premise Agentic AI Workbench using Open-Weight Multimodal LLMs for Confidential Industrial Work*
Proposing organisation: MRPL (Mangalore Refinery and Petrochemicals Limited)

OUR BUILD

BiMax — an air-gapped agentic workbench. ~127,000 lines of TypeScript across 705 modules, a Go/Bubble Tea terminal front-end, 43 agent tools, 263 test suites.

EVENT

24-hour GITAM Hyderabad Internal SIH · 4–5 September 2026 · G412 Multipurpose Hall & G413 Makerspace

INSTITUTIONAL SPOC

Mr. Fakhruddin Sheik, Deputy Director – VDC

PURPOSE OF THIS DOCUMENT

Every team member reads this. It contains the rules we are judged against, the problem we chose, exactly what we built and how it works, an honest statement of what is and is not finished, a minute-by-minute demo script, a slide-by-slide deck blueprint, and the question bank judges will draw from.

How to use this document

This pack has two jobs. Read it end to end once so you can answer for the project in front of a judge. Then feed it to an LLM to generate the deck.

If you are a team member

1. Read **Part A** (30 min) — the rules and what judges score. This decides how we present, not what we built.
2. Read **Parts B–C** (30 min) — the problem and our answer to it. Everyone must be able to give this in 90 seconds without notes.
3. Read **Part D** carefully for *your* subsystem, and skim the rest. You are not expected to know every file. You are expected to know how the system fits together.
4. Read **Part G** (honest status) — this is the most dangerous section to be caught not knowing. A judge who finds a gap you did not disclose stops believing everything else.
5. Rehearse against **Part I** (question bank) with a teammate, out loud.

If you are generating the deck

Upload this PDF to ChatGPT / Claude / Gemini and paste this prompt:

```
You are helping a 6-person team prepare a 10-minute pitch for the GITAM Hyderabad Internal Smart India Hackathon, for SIH problem statement PS 26117 (MRPL) – "Sovereign On-Premise Agentic AI Workbench using Open-Weight Multimodal LLMs for Confidential Industrial Work".
```

```
The attached PDF is the complete, factual reference for our project. Every technical claim in it is backed by shipped code. Do not invent features that are not in it, and do not soften the "Honest status" section – our credibility depends on it.
```

```
Build the slide deck described in Part H of the PDF. For each slide give me:
```

1. the headline (max 8 words),
2. 3–5 bullets of no more than 12 words each,
3. speaker notes of 60–90 spoken words,
4. a one-line description of the visual on that slide.

```
Constraints:
```

- The judging criteria are in Part A. Weight the deck toward clarity, feasibility, impact and UX – not toward code volume.
- A judge must understand the problem, why it matters, what we built, and why it is different, within the first 2 minutes.
- Use plain industrial language. Assume the judge knows engineering but not LLMs.
- Keep the total speaking time under 10 minutes with 3 minutes of demo inside it.

Then follow up with: *“Now write the 3-minute demo narration from Part F, as a word-for-word script.”*

Contents

- **Part A** — The event, the rules, and what judges actually score
- **Part B** — PS 26117 decoded: what MRPL is actually asking for
- **Part C** — Our solution in one page
- **Part D** — Architecture and implementation, subsystem by subsystem
- **Part E** — The sovereignty mechanism, in depth (this is our novelty)
- **Part F** — The demo: minute by minute

- **Part G** — Honest status: built / partial / not started
- **Part H** — Slide-by-slide deck blueprint
- **Part I** — Question bank and per-member ownership
- **Part J** — Glossary for non-specialists

Part A — The event, the rules, and what judges score

A.1 What is happening

Item	Detail
Event	GITAM Hyderabad Internal Smart India Hackathon
Dates	4–5 September 2026
Duration	24 hours, continuous
Venue	G412 Multipurpose Hall + G413 Makerspace
Status	Mandatory. A team that does not clear this round cannot progress toward national SIH.
Institutional SPOC	Mr. Fakhruddin Sheik, Deputy Director – VDC

THE SELECTION PIPELINE

GITAM screening → **24-hour Internal SIH (this event)** → nomination / secondary selection with the SPOC → SIH national process → Grand Finale.

READ THIS TWICE

Winning the internal round does **not** put us in the Grand Finale. GITAM has a limited number of nominations it can send forward. The internal round is an argument for why we should be one of them. That framing should shape every slide.

A NOTE ON THE SOURCE DOCUMENT

The guidelines describe “three phases”; the fourth numbered item is the registration deadline, not a competition phase. The orientation workshop time is printed as “11pm–12pm”, which is internally inconsistent — that event has already passed (31 August), so it does not affect us.

A.2 Team composition rules

- **Exactly 6 students**, including the team leader.
- All six from **GITAM Hyderabad**.
- **At least 1 female team member is compulsory.**
- Mixed branches are allowed and encouraged (e.g. 3 CSE + 1 AI/ML + 1 ECE + 1 management).
- For a software problem statement, most members are expected to have programming skills.
- One designated **team leader**, responsible for coordination and communication.

A.3 Documents — do not lose points here

The guidelines are explicit: “*Teams failing to submit required documents may not be shortlisted/confirmed.*” This is the cheapest possible way to be eliminated.

Document	Who	Notes
HOD Consent Letter	Per department represented	Departmental approval to participate
Parent Consent Form	Per participant	Parent permission for the overnight event
Undertaking Form	One per team	Signed by the Team Leader ; commitment to follow SIH rules
College ID Card	Every participant	Carry the physical card, not a photo
G-Residence full-day leave	Hostel residents only	Apply stating attendance at the 24-hour internal hackathon on campus

A.4 The ten judging criteria

This table is the specification for our presentation. Every slide should be traceable to a row in it.

Criterion	What the judge is asking	Where we answer it
Novelty	Is this actually different, or a wrapper around an existing product?	Part E — egress fails closed at the Node primitives, and the proof is an artefact, not a promise
Complexity	Is there meaningful engineering here?	Part D — 43 tools, a 13-stage guard pipeline, sandboxed execution, code graph, multi-agent orchestration
Clarity	Can you explain the problem and the solution?	Parts B and C — must be deliverable in 90 seconds by any member
Details	Have you thought through how it works?	Part E — why classification is syntactic and never DNS-resolved, and what that defends against
Feasibility	Can this realistically be built?	It is built and running. Part G states exactly how much.
Practicability	Can someone actually deploy this?	Runs on one workstation with Ollama; local-provider presets ship in the binary
Sustainability	Does it survive past the hackathon?	263 test suites, an executable product boundary test, CI on macOS and Linux
Scale of impact	Who benefits, and how many?	Part B — every refinery, defence unit, hospital and PSU that cannot send data to a cloud model
User experience	Is it usable, not just clever?	Single self-contained binary, one command, a terminal UI with an inline audit report
Future progression	Can this become a product?	Part G — the S5–S8 roadmap: org knowledge base, multi-user server, RBAC, domain templates

THE STRATEGIC CONCLUSION

Judges are **not** scoring lines of code. A smaller, clearly-explained system beats a larger, badly-explained one. Six of the ten criteria — clarity, details, feasibility, practicability, UX, scale of impact — are won by *explanation*, not by implementation. Our implementation is already deep; our risk is failing to communicate it. Lead with the problem, show it working, and hold the technical depth in reserve for the Q&A.

A.5 The mentor

The mentor is not a signature on a form. Under the rules they are expected to guide the team, work with us during the internal hackathon, support prototype development, and potentially continue with the team through to the Grand Finale. **We are permitted to discuss architecture, strategy and implementation problems with the assigned mentor during the event.** Use that — a mentor who has heard our pitch twice will catch the gap a judge would have found.

A.6 How to spend the 24 hours

The judging criteria include feasibility, UX, clarity and impact. None of those improve by adding twenty more features. The highest-value allocation:

Window	Focus	Owner
Hour 0–2	Lock scope. Decide what the demo shows and freeze it. Write the demo script first, then make it true.	Leader + all
Hour 2–10	Make the demo path work end to end, offline, on the actual demo machine. Nothing else matters until this is green.	Engine + integration
Hour 10–14	Deck v1 from Part H. Parallel with engineering, not after it.	Presentation owner
Hour 14–18	Failure-proof the demo: rehearse it cold, on battery, with Wi-Fi physically off, twice.	Demo owner
Hour 18–21	Rehearse the pitch aloud, full team. Run the Part I question bank at each other.	All six
Hour 21–24	Recordings and fallbacks. Buffer. Sleep in shifts.	All six

THE SINGLE BIGGEST RISK

A live demo that fails. Record a clean screen capture of the full demo in hour 14 and keep it on the presenting laptop. If the live run breaks, switch to the recording in one keystroke and keep talking. Never debug in front of a judge.

Part B — PS 26117 decoded

“Sovereign On-Premise Agentic AI Workbench using Open-Weight Multimodal LLMs for Confidential Industrial Work.” Proposed by MRPL — Mangalore Refinery and Petrochemicals Limited.

B.1 The problem in industrial terms

A refinery runs on documents. Inspection reports, standard operating procedures, piping and instrumentation diagrams, engineering calculations, vendor correspondence, approval notes. Many of them are scanned. Many are hand-annotated. Almost all of them are confidential — commercially, and often under statute.

Modern AI assistants would help enormously with this work. Engineers cannot use them, because using them means uploading the document. For a refinery, a defence unit, a hospital or a bank, that upload is not a policy preference — it is a disqualifying event. So the highest-value AI use case in the organisation is the one it is structurally forbidden to adopt.

THE ONE-SENTENCE PROBLEM
The organisations with the most to gain from agentic AI are exactly the organisations that cannot let their data leave the building.

B.2 What the problem statement demands

Requirement	What it means concretely
Model-agnostic backend	Run many open-weight models, swap them without redesigning the system
Automatic model selection	Pick the right model per task — a coding model for code, a vision model for a scanned drawing
New models addable without redesign	Adding a model is configuration, not a code change
Agentic multi-step work	Plan, act, observe the result, iterate — not single-shot question answering
Local tools	Read and write files, search, run git, query code — on the machine
Sandboxed code execution	The agent can run code, under kernel-enforced isolation
Real deliverables	Actual Word / Excel / PowerPoint / PDF files, not markdown in a chat box
Local knowledge grounding	Answer from the organisation's own manuals and SOPs, with citations
Multimodal input	Scanned PDFs, engineering drawings, handwriting
Runs entirely on-premises	And proves it — the PS asks to “show, through logs or a visible network monitor, that no external calls are made at any point.”

THE LOAD-BEARING ROW
The last row is the whole problem statement. Every other requirement has a dozen products that satisfy it. **The proof of zero egress is the differentiator**, and the PS explicitly asks for it as a deliverable. That is why our architecture treats it as the primary feature and not as a setting.

B.3 Why existing approaches fail

Approach	Why it fails this problem statement
Cloud AI assistants (ChatGPT, Claude, Copilot)	The document leaves the premises. Disqualifying, regardless of the vendor's contractual assurances.
“Enterprise” / VPC cloud tiers	Moves the trust boundary, does not remove it. Still an external network dependency, still auditable only through the vendor.
A local model in a chat window (Ollama + a UI)	Answers questions. Cannot act — no file editing, no sandboxed execution, no git, no document generation. Not agentic.
Local agent frameworks (LangChain, AutoGen, and similar)	Libraries, not products. And critically: nothing prevents a single dependency from opening a socket . Telemetry, an SDK's update check, a browser automation library — one forgotten call site and the sovereignty claim is false, silently.
Per-host consent prompts	Answers “did the user agree?”. An air-gapped deployment is audited on “did anything leave?”. Those are different questions.

THIS IS THE ARGUMENT THAT WINS THE ROUND

Everyone at this hackathon can wire a local model to a chat UI. The hard, unsolved part is **proving** that a 700-module application with dozens of third-party dependencies made zero external calls — continuously, with the caller and purpose attached, in a form an auditor can read. That is what we built, and it is the part nobody else will have.

Part C — Our solution in one page

Everyone on the team must be able to say this from memory. If you only learn one page of this document, learn this one.

THE 90-SECOND PITCH

The problem. A refinery's most valuable documents are the ones it is forbidden to upload. So the engineers who would benefit most from AI are the ones who can't use it.

What we built. BiMax is an agentic AI workbench that runs entirely on the machine in front of you. It plans, reads and writes files, runs code in a kernel sandbox, reads scanned documents with on-device OCR, and produces real Word, Excel, PowerPoint and PDF deliverables — driven by open-weight models served locally.

Why it is different. Everyone can point a chat window at a local model. We made the *absence* of network traffic a machine-checked, auditable property. We patch the network at the language runtime itself — every `fetch`, every socket, every DNS lookup, from every dependency — and write an append-only ledger of every outbound attempt, allowed and refused, with the calling subsystem attached. The problem statement asks us to prove no external calls are made. We hand the auditor the proof.

Show it. We unplug the network, run a full agentic task end to end, then print the ledger: `142 attempts · 142 loopback · 0 external`.

C.1 What the product is

- A **single self-contained binary**. A Go / Bubble Tea terminal application with the agent engine embedded. The target machine needs no Node, no Bun, no dependency install — which matters when the target machine is an air-gapped server that cannot reach a package registry.
- A **provider-agnostic engine** speaking the OpenAI-compatible protocol, with built-in presets for Ollama, vLLM, LM Studio and llama.cpp — all pointed at loopback.
- An **agent loop** with 43 tools: file editing, AST-addressed symbol editing, search, git, LSP, a live code graph, sandboxed shell, document reading and document writing, sub-agent spawning, planning and memory.
- A **sovereignty perimeter** that fails closed and records everything.

C.2 The six things that make it credible

Claim	The mechanism behind it
It really is offline	Egress is refused at the Node runtime primitives — <code>fetch</code> , <code>http.request</code> , <code>https.request</code> , <code>net.connect</code> , <code>tls.connect</code> , <code>dns.*</code> — patched at boot, so a dependency that never heard of our guard is still caught.
You can prove it	An append-only NDJSON ledger records every attempt: timestamp, host, classification, verdict, calling subsystem, purpose. Allowed calls too — a ledger of refusals proves nothing about what was allowed.
It really acts	43 tools with a fixed 13-stage guard pipeline in front of every one of them, plus 5 write-path integrity guards inside the editing tools.
It is safe to let loose	Shell runs under a kernel sandbox — macOS Seatbelt or Linux bubblewrap. In sovereign mode the sandbox is forced on with the network denied at the kernel, and if no OS backend exists the shell tool is refused rather than run unprotected .
It reads scanned paper	PDF text layer first; on-device OCR only for pages that have none. Apple Vision on macOS, Tesseract on Linux. If neither is available it reports failure — it never returns empty text that would make a scanned report look blank.
It produces real work	Native <code>.docx</code> , <code>.pptx</code> , <code>.xlsx</code> and <code>.pdf</code> writers, exposed to the agent as a tool.

C.3 The flagship scenario

Spoken as a story, this is the demo:

1. An inspection engineer has a **scanned inspection report** — a photograph of a paper form with handwritten annotations.
2. They drop it into BiMax on their own workstation, with the network cable out.
3. The router sees a document attachment and selects the **vision-capable local model**, not the coding model.
4. The ingestion path checks for a text layer, finds none, and rasterises the pages to on-device OCR.
5. The agent extracts the findings, **each with a page citation**, so a reader can go back to the source.
6. It drafts an **approval note as a real .docx** .
7. We run `/sovereign report` . Zero external destinations, ever contacted, by this installation.

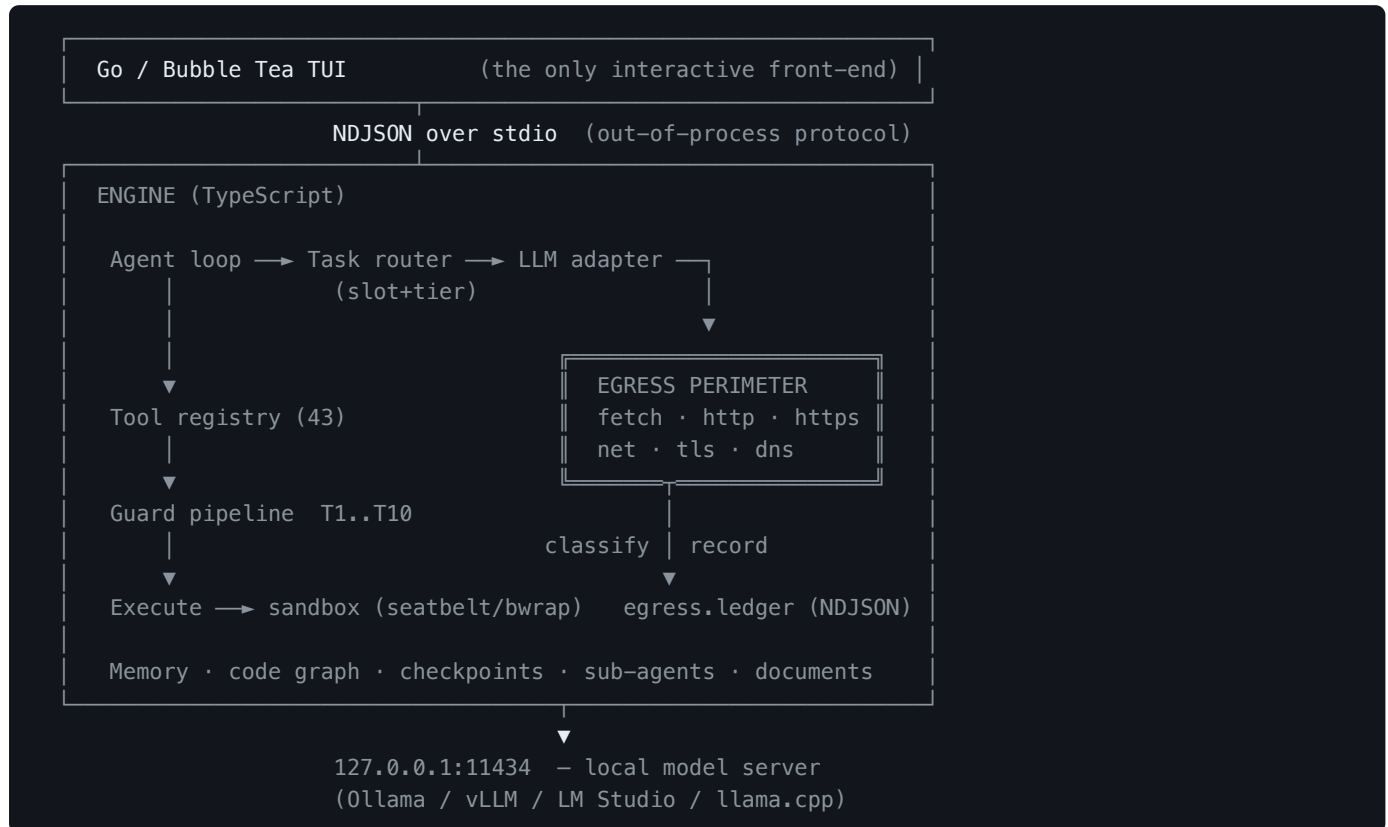
WHY THIS SCENARIO AND NOT ANOTHER

It exercises every requirement in the problem statement in one continuous run — multimodal input, automatic model selection, agentic multi-step work, a real deliverable, and the egress proof — and it is a task an MRPL engineer does every week. Judges remember a story about a person doing a job. They do not remember a feature list.

Part D — Architecture and implementation

Scale of the codebase: 705 TypeScript modules, ~127,000 lines in the engine; a Go / Bubble Tea front-end; 43 agent tools; 263 test suites. This part walks the system in the order data flows through it.

D.1 The shape of the system



D.2 The agent loop — `src/core/agent.loop.ts`

The heart of the system. One turn is: stream the model's response, surface visible tokens and hidden reasoning separately, execute any tool calls the model requested, feed the results back, and iterate up to a configurable maximum. Concurrency-safe tools run in parallel; destructive ones run sequentially.

Three details worth knowing, because they are the difference between a demo and a product:

- **Output-contract sanitiser** — strips leaked filler like “*No function call is needed...*” and reports when a turn was *only* filler, so the loop can regenerate rather than showing the user nothing.
- **Text tool-call recovery** — weaker open-weight models sometimes emit a tool call as plain JSON in the message body instead of in the tool-call channel. The parser recovers it. This matters specifically because we are running small local models, not frontier cloud models.
- **Loop detection and failure memory** — the agent notices when it is repeating itself and when a previously-tried approach already failed.

D.3 Model access — `src/core/llm.adapter.ts`, `src/cli/provider.ts`

One OpenAI-compatible client serves every provider, which is what makes the system model-agnostic in the sense the PS asks for: adding a backend is a preset, not a rewrite.

Local preset	Endpoint	Default model
Ollama	<code>http://127.0.0.1:11434/v1</code>	<code>qwen2.5-coder:7b</code>
vLLM	<code>http://127.0.0.1:8000/v1</code>	<code>Qwen/Qwen2.5-Coder-7B-Instruct</code>
LM Studio	<code>http://127.0.0.1:1234/v1</code>	<code>qwen2.5-coder-7b-instruct</code>
llama.cpp server	<code>http://127.0.0.1:8080/v1</code>	<code>local-model</code>

All four are **keyless** — they carry a placeholder bearer token because the OpenAI SDK rejects an empty one. There is a deliberate constraint here worth quoting in Q&A: *a keyless local preset must never be allowed to rescue an unkeyed cloud provider in the key-pool builder*, or selecting “OpenAI” with no key would silently serve answers from the local box and the user would never know which model answered.

Also in this layer: **capability shaping** (`src/core/capabilities.ts`) strips parameters a given model does not serve — reasoning effort, parallel tool calls, fixed-sampling temperature — before the request is sent, rather than letting the server reject it. New models get a capability row; unmeasured models resolve to a conservative floor.

D.4 Task routing — `src/cli/task.router.ts` SHIPPED

The PS asks the system to “pick the right model for a given task”. That is two independent questions, and we route on both axes:

Axis	Question	Values
Weight (<code>model.router.ts</code>)	How much model does this turn deserve?	<code>lite</code> / <code>heavy</code> — a cost and latency decision
Slot (<code>task.router.ts</code>)	What <i>kind</i> of model does this turn need?	<code>chat</code> / <code>code</code> / <code>vision</code> / <code>draft</code>

The slot axis exists because a turn carrying a scanned P&ID does not need a *bigger* text model, it needs a *different* model. Before this shipped, an image turn was served by whichever text model the weight axis happened to pick.

THE DESIGN CONSTRAINTS, AND WHY THEY MATTER

- **Fully local, deterministic, zero model calls.** Routing never adds a model call to the turn path. An earlier LLM-based classifier cost 1.17 s per turn and was removed on that evidence. Routing cannot block or time out a turn.
- **Most-specific slot first.** The signals overlap. “*Read this scanned inspection report and draft an approval note*” is simultaneously a vision turn and a drafting turn. **Vision wins**, because a model that cannot see the report cannot draft anything from it. A missing modality is a hard failure; a deliverable format is not.
- **Vision is decided from the file extension, not the prose.** A user who drags in a drawing and types “what is this?” gives a text classifier no signal at all. PDF and TIFF are treated as vision inputs because a scanned document reaches the model as rasterised pages.
- **Deliberately narrow keywords.** The word “diagram” alone does not trigger the vision slot — “*explain the architecture diagram in the README*” is a text turn about a described diagram, and routing it to a vision model with nothing to look at would be strictly worse.

The slot metadata was already in the tree and simply unread: every row in the model catalogue declares what it is `recommendedFor` . The router supplied the missing half — deciding which slot a turn belongs to.

D.5 The tool surface — 43 tools

Tools are what makes this *agentic* rather than conversational. The registry holds them; a factory builds each one with governor gating, destructive and concurrency flags, and a typed JSON schema with per-parameter descriptions.

Domain	Tools
Editing & files	<code>read</code> , <code>write</code> (auto-backed-up), <code>edit</code> (exact search/replace), <code>multiedit</code> (atomic multi-file batch), <code>syboledit</code> (AST-addressed edit by symbol name), <code>delete</code> , <code>mkdir</code> , notebook cell edit
Search & navigation	<code>grep</code> , <code>glob</code> , <code>cd</code> , graph query, graph context, LSP query, scout (fast read-only recon), tool search
Execution & VCS	<code>bash</code> (risk-classified), <code>git</code> (status/diff/log/add/commit — never push), related-tests
Documents	<code>read_document</code> (PDF/scan ingestion + OCR), <code>document</code> (write <code>.docx</code> / <code>.pptx</code> / <code>.xlsx</code> / <code>.pdf</code>)
Memory & context	<code>remember</code> , memory query (semantic recall), free-context (drop stale context under pressure), goals
Sub-agents & planning	<code>spawn subagent</code> , tasks (manage/kill live sub-agents), <code>plan</code> , <code>ask-user</code>
Extensions	<code>todo</code> , skills (use/install/author), MCP management, register-agent, mode, model management, workspace
Web	<code>web fetch</code> (SSRF-guarded), <code>web search</code> — both refused in sovereign mode

A DETAIL WORTH CITING UNDER “DETAILS”

Mutating and executing tools return **typed outcomes** (an ok/error object with a failure class), not free text. That is what lets the system attribute results, detect loops, and decide whether an edit is verified. Read-only meta tools return plain strings by design — the typed contract is deliberately scoped to actions whose outcomes are attributed, not applied universally for symmetry.

D.6 The guard pipeline

Every tool call the model makes passes through **one fixed, ordered pipeline** — not a scattering of ad-hoc checks. This is centralised in the tool factory, so the sequence is the same for all 43 tools and there is one place to audit.

BEFORE A MODEL CALL

#	Guard	Purpose
R1	Capability shaping	Strip parameters the target model does not serve, before send
R2	Budget veto	Hard daily spend cap, mutex-guarded reserve → record → release, persisted, per-day reset
R3	Headroom compression	Gated to token pressure only; code-guarded so source is restored verbatim

ON EVERY TOOL CALL

#	Guard	Applies to
T1	Bypass short-circuit	All, in bypass mode
T2	Plan-mode block	Mutating tasks while in plan mode
T3	Taint restriction	Shell commands, once untrusted content has entered the context — prompt-injection containment
T4	Persistent rules	Allow/deny by task type; taint cannot be waived by a rule
T5	Filesystem veto	Writes and deletes — workspace confinement, forbidden paths and extensions
T6	Bash static analysis	Tree-sitter risk classification; read-safe commands auto-approve
T7	ML risk classifier	Ambiguous shell commands in auto mode
T8	Interactive prompt	Destructive tasks, or strict mode
T9	Pre-tool hooks	All — user hooks may block
—	<i>tool executes</i>	
T10	Post-tool hooks	All — may append feedback, e.g. a type-check result

INSIDE THE WRITE TOOLS

#	Guard	Purpose
W1	Blast-radius gate	Confirm before overwriting a graph-critical symbol (opt-in)
W2	Diff approval	Inline diff confirmation
W3	Corrupt-write guard	Refuse a flattened / newline-stripped full-file overwrite
W4	Edit Shield	Refuse a write that introduces <i>new</i> syntax errors versus what is on disk
W5	Transaction + backup	Snapshot pre-write state, enabling undo and rollback

Permission, integrity, approval and rollback are four different jobs — which is why none of these five is redundant with the governor above it. Per-guard wall-time is accumulated and reportable, so a slow guard is visible before it is blamed.

D.7 Sandboxed execution — `src/sandbox/exec.sandbox.ts`

The agent can run shell commands. That is only acceptable if the blast radius is bounded by something stronger than the agent's own good intentions, so shell runs under a **kernel sandbox**: Seatbelt on macOS, bubblewrap on Linux, with write and network floors.

THE SOVEREIGN INTERACTION — SAY THIS TO A JUDGE

A child process has its own network stack. Our in-process perimeter cannot see it. So in sovereign mode the sandbox is **forced on** with the network denied at the kernel — `(deny network*)` on macOS, `--unshare-net` on Linux — and if no OS sandbox backend is available the shell tool is **refused rather than run without kernel isolation**. We state the scope limit rather than hiding it, and then we close it at a different layer.

D.8 Document ingestion and deliverables

READING — SRC/DOCUMENTS/PDF.RASTER.TS , OCR.TS , READDOCUMENTTOOL SHIPPED

- **Text layer first.** A PDF that already has selectable text is never sent to a vision model. OCR runs only on pages that have no text layer. This is a large cost and latency saving on a single mid-range GPU.
- **Two OCR backends, chosen by availability, not preference.** macOS ships Vision — already on the machine, no weights to download, and materially better at **handwriting**, which matters when the input is a hand-annotated inspection note. A refinery's GPU server is a Linux box, so Tesseract is the second backend. Both run entirely offline.
- **The macOS helper is compiled on first use, not shipped.** Vision is only reachable from a native process, so a ~90-line Swift helper is compiled locally with `swiftc` (part of the Command Line Tools) and cached. Nothing is fetched. This keeps a binary out of the repository and keeps the helper's source reviewable beside the code that calls it.
- **Failure is reported, never silently degraded.** If neither backend is available, the tool says so. An OCR layer that quietly returned empty text would make a scanned report look like a blank one — and the model would then confidently summarise a document it never read. That failure mode is worse than an error.

WRITING — SRC/DOCUMENTS/*.WRITER.TS SHIPPED

Native writers for `.docx` , `.pptx` , `.xlsx` and `.pdf` , with a shared house-style module, exposed to the agent as a single document tool. The agent produces the file an engineer actually has to hand to someone — not markdown they then have to reformat.

D.9 Memory, retrieval and the code graph

Layer	What it does
Short-term memory	Working message buffer with pruning that preserves system messages
Context manager	Token accounting and auto-compaction at ~70% capacity — summarises older turns <i>without splitting a tool-call/result pair</i> , and compacts reactively on a context-length error
Hybrid retrieval	BM25 lexical + vector similarity, combined by a fusion stage, then reranked
Project memory	Durable per-project conventions, decisions and gotchas, injected into each turn's system prompt
Code graph	Persistent nodes and edges parsed from source by a static analyser; kept fresh by an observer as files change
Impact engine	Blast-radius analysis — what depends on this symbol or file. Powers the confirm-before-overwrite gate above.
Cognitive graph	Answers architecture questions from the graph rather than from the model's guess

KNOWN GAP — DISCLOSE THIS, DO NOT HIDE IT

The embedding backend is remote-first and returns `null` when unavailable, so callers degrade to lexical BM25 search. Air-gapped, that means the “semantic” knowledge base **silently becomes keyword search**. Closing this (local embeddings via the local runtime, a local cross-encoder reranker, and a visible degraded-mode banner) is work package 6 and is **not done**. See Part G.

D.10 Multi-agent orchestration

All of these run in **isolated git worktrees**, so the user's branch is never touched until they merge. Sub-agents run as worker threads with a watchdog timeout, so a hung worker cannot block the parent.

Capability	What it does
Swarm	Decompose a task into a dependency graph, run topological “waves” of parallel workers, merge to an integration branch
Speculate	Propose N distinct approaches, implement each in parallel, test them, recommend the winner
Council	Run rival agent CLIs on the same task, judge by the project's own test suite, keep the winner
Heal	Run the tests; if red, a fix agent iterates in a worktree until green, then surfaces the patch on a branch
Checkpoint / rewind	Snapshot and restore the working tree — manual and automatic

Note for the pitch: these are impressive but they are *not* what PS 26117 asks for. Mention them once as evidence of depth; do not spend demo time on them.

D.11 The front-end

A Go / Bubble Tea terminal UI. The engine talks to it out-of-process over an NDJSON stdio protocol, so the front-end can be replaced without touching engine logic — the seam that would let a browser client be added for the multi-user server deployment later.

- Committed output is flushed to the terminal's **native scrollbar**; only the live region re-renders. Scrolling and history belong to the terminal, where the user already knows how they work.
- Colourised diff cards, fixed-slot tool cards, markdown rendering, search.
- Nine themes including **daltonised** and ANSI variants; documented support for `NO_COLOR`, reduced motion, contrast and screen readers.

THIS IS A USER EXPERIENCE POINT, AND UX IS A SCORED CRITERION

The delivery format is a decision, not an accident. A refinery's air-gapped server is administered over SSH. A terminal binary with no runtime dependencies installs and runs there; an Electron desktop app or a web stack does not. Accessibility support is shipped rather than promised.

Part E — The sovereignty mechanism

This is our novelty claim and it will carry the Q&A. Whoever presents must understand this part completely. It is four modules: `sovereign.ts` (decide), `egress.guard.ts` (enforce and record), `egress.ledger.ts` (persist), `egress.perimeter.ts` (intercept).

E.1 The insight: consent is not proof

Before this work, the codebase asked the user for permission per host. That answers “**did the user agree?**” An air-gapped deployment is audited on “**did anything leave?**” Those are different questions, and only the second is what the problem statement asks us to demonstrate.

So sovereign mode makes external egress fail **closed** at every seam, and writes every attempt — allowed and refused alike — to an append-only ledger.

E.2 The discovery that shaped the design

THE CENSUS — TELL THIS STORY TO A JUDGE

We built a single guard function and wired it in at the call sites we knew about. Then we counted the call sites. **The guard had exactly two callers.** Meanwhile **sixteen modules opened sockets directly** — the browser automation library, the remote embedding and rerank backends, the MCP client, the GitHub API client, the self-updater, and a telemetry network probe that performed DNS, TCP and TLS to the provider's origin. That last one is the exact leak the module's own documentation warns about.

An opt-in guard is not a perimeter. A perimeter that a dependency can forget to call is decoration. So we moved the guard *beneath* the code.

E.3 The perimeter — `src/security/egress.perimeter.ts`

At boot, from all three entrypoints, we patch the Node runtime's own networking primitives:

```
fetch          // the modern path – patched async, so a refusal rejects the promise
http.request    // throws synchronously, on purpose
https.request
net.connect     // raw TCP
tls.connect     // TLS
dns.*           // name resolution is itself egress
```

A dependency that has never heard of our guard is caught anyway, because it cannot open a socket without going through one of these. That is the difference between a policy and a perimeter.

FIVE IMPLEMENTATION DETAILS THAT ARE WORTH POINTS UNDER “DETAILS”

1. **Patch the object that `require('http')` returns, not an ES-module namespace.** A namespace import gives you getter-backed copies; assigning to them throws.
2. **Every layer verifies; only the outermost records.** One logical `fetch` descends through TLS, TCP and DNS. Without this, a single request would appear in the audit four times and the headline number would be meaningless. The verdict is always computed at every layer — only the *record* is suppressed, so a suppressed inner layer can still refuse.

- 3. **Deduplication is keyed on the host, not the raw target.** `net.connect` sees `example.com:443` ; the `dns.lookup` beneath it sees `example.com` . Keying on the raw string would fail to match them.
- 4. **Unix domain sockets are skipped.** A socket path is not network egress, and refusing it would break local container tooling.
- 5. **Caller attribution reads the stack** — a source path first, then falling back to the **function name**. The shipped engine is a single-file compiled executable with no source paths in its stack traces; without the fallback, every ledger line would read `subsystem: "fetch"` and the audit would name no culprit.

E.4 Classification — `src/security/sovereign.ts`

Four classes. `external` is the only refusable one.

Class	Covers
<code>loopback</code>	IPv4 and IPv6 loopback
<code>private-lan</code>	RFC1918, CGNAT, IPv6 unique-local and link-local, <code>.local</code> / <code>.internal</code> suffixes
<code>allowlisted</code>	Operator-declared on-premises hosts that carry ordinary names
<code>external</code>	Everything else — including a bare single-label hostname like <code>gpu01</code> , which we genuinely cannot prove is local

WHY WE NEVER RESOLVE DNS TO CLASSIFY — THE BEST ANSWER IN OUR Q&A

The obvious implementation resolves the hostname and checks whether the address is private. It is wrong twice.

1. **A resolution is itself egress.** Asking a resolver about `secret-project.example.com` leaks the name to whoever runs that resolver — on an air-gapped site, the exact leak we exist to prevent. *A check that leaks in order to decide whether leaking is allowed is not a check.*

2. **The answer can change between the check and the connection.** A name that resolved to `10.0.0.5` during the check can resolve to a public address microseconds later — classic DNS rebinding. Anything derived from the first answer is a claim about the past, not about the connection you are making.

So classification is **syntactic**, and **fails towards refusal** when it cannot prove locality.

ADVERSARIAL ADDRESS SPELLINGS

Address strings are hostile. Every one of these is a real evasion:

Written as	Actually connects to
<code>http://2130706433/</code>	127.0.0.1 (integer form)
<code>http://0x7f.1/</code>	127.0.0.1 (hex form)
<code>http://010.0.0.1/</code>	8.0.0.1 — octal. It <i>looks</i> like a private 10.x address and is not.
<code>[::ffff:127.0.0.1]</code>	Stored as <code>[::ffff:7f00:1]</code>

A hand-rolled dotted-quad check gets every one of these wrong, and at least half of them wrong in the dangerous direction. So we push every target through the **WHATWG URL parser** — the same canonicalisation the fetch stack applies before it connects — and classify the parsed `hostname` . Bare `host:port` forms are re-parsed as `http://host:port` first. *That is the difference between checking the request and checking a description of the request.*

The residual limitation is stated rather than hidden: an allowlisted name whose DNS points off-site would be permitted. That is an operator decision, and it is recorded in the ledger.

E.5 The ledger — `src/security/egress.ledger.ts`

Append-only NDJSON. One line per outbound attempt: timestamp (ISO-8601 UTC), host, classification, verdict, calling subsystem, purpose, and whether sovereign mode was active for that attempt.

THREE DESIGN DECISIONS, EACH DEFENSIBLE UNDER QUESTIONING

- **Allowed calls are recorded too.** A ledger of refusals answers “what did we block?” — a question nobody asks at an audit. The question actually asked is “*what did you send, and to whom?*” So an allowed loopback call to the local model server is recorded with the same weight as a refused external one, and the headline number is honest by construction.
- **The guard records even when sovereign mode is off.** A user who has never enabled the mode can still ask “what did it contact during that task?” and get a complete answer. Enabling the mode later changes the *verdict*, not the *visibility*.
- **One line per attempt, appended, never rewritten.** A truncated write costs the last line, not the file.

SAY THIS BEFORE A JUDGE SAYS IT TO YOU

This is **tamper-evident at the application level, not tamper-proof**. Anything with write access to the file can edit it. Claiming otherwise would be exactly the species of overstatement this module exists to prevent. Making it genuinely append-only is the operating system's job — `chattr +a`, a log shipper, or an audit mount — and belongs in the deployment guide. The file is created mode 0600 inside a 0700 directory, because it records hostnames and purposes drawn from confidential work.

Volunteering this limitation is worth more than concealing it. It signals that the rest of our claims were audited with the same honesty.

E.6 The operator surface — `/sovereign`

The problem statement asks us to *show* it. This is where an evaluator reads the answer. Also available as a `--sovereign` launch flag and a boot banner.

Command	Output
<code>/sovereign status</code>	Mode, whether the perimeter is installed and across which primitives, shell posture, allowlist, this session's tally, ledger path
<code>/sovereign on / off</code>	Toggle fail-closed enforcement
<code>/sovereign report</code>	Full audit: session tally, on-disk tally, every external host ever contacted with attempt and refusal counts and the subsystems responsible, and the last 12 attempts in full
<code>/sovereign json</code>	Machine-readable form, for a compliance pipeline
<code>/sovereign allow <host></code>	Declare an on-premises host that carries an ordinary name
<code>/sovereign check <host></code>	Classify a destination without contacting it

WHAT /SOVEREIGN STATUS ACTUALLY PRINTS

```
Sovereign mode: ON – external egress fails closed
Egress perimeter: installed (fetch · http · https · net · tls · dns)
Shell:           network denied at the kernel (seatbelt)
Allowlist:       (empty – only loopback and private-LAN are local)

This session:    142 attempts · 142 loopback · 0 external

Ledger: ~/.../egress.ledger
```

WHAT A REFUSAL LOOKS LIKE IN THE REPORT

```
EGRESS REPORT

This session: 7 attempts · 5 loopback · 2 external
On disk:      149 attempts · 147 loopback · 2 external

External destinations contacted (1):
  integrate.api.nvidia.com – 2 attempts, 2 refused · from LlmAdapter, embedBatch

Last 2 attempts:
  2026-09-03T19:41:08Z  REFUSED  external  integrate.api.nvidia.com  (LlmAdapter chat)
  2026-09-03T19:41:09Z  REFUSED  external  integrate.api.nvidia.com  (embedBatch)

Scope: this records what the APPLICATION attempted. A child process has its own
network stack and is governed by the sandbox, not by this ledger.
```

That refusal is **live-verified**: the compiled engine under `--sovereign` refused the cloud provider from both the embedding path and the model adapter, and both were recorded. It is a real transcript, not a mock-up — which is why we can run it on stage.

E.7 The three layers, stated together

Layer	Covers	Mechanism
In-process	The engine and every one of its dependencies	Runtime primitive patching — fail closed, always recorded
Child processes	Anything the shell tool runs	Kernel sandbox with the network denied; refuse the tool if no backend exists
The machine	Everything else	Out of scope, and we say so. The deployment answer is an air-gapped host; the demo answer is unplugging the cable.

THE SENTENCE TO END THE TECHNICAL Q&A ON

“We did not make a promise that nothing leaves. We made it a property the machine checks, at the lowest layer we control, and we hand you the log.”

Part F — The demo, minute by minute

Three minutes, inside a ten-minute pitch. Rehearse it until it needs no thought. The demo owner drives; the presenter narrates; nobody else touches the laptop.

F.1 Pre-flight — done before we are called in

- Local model server running and warm (Ollama with a coding model and a vision model already pulled and loaded — **a cold model load will eat the whole demo**).
- Sample scanned inspection report already on the machine, at a short path you can type without looking.
- Terminal font size raised to be readable from the back of the room. Test it from the back of the room.
- Do-not-disturb on. Notifications off. Battery charged.
- **The recording of this exact demo is open in a second window, one keystroke away.**

F.2 The run

Time	Action	What the presenter says
0:00	Run <code>/sovereign status</code>	“Before anything runs — this is the perimeter. It's installed across fetch, http, https, raw TCP, TLS and DNS. Shell is sandboxed with the network denied at the kernel. Zero external calls so far.”
0:20	Physically unplug the network / switch Wi-Fi off, visibly, on camera.	“From here on, there is no network. Everything you see is running on this machine.”
0:35	Hand it the scanned inspection report	“This is a scanned inspection report with handwritten annotations. It has no text layer — it's a photograph of paper.”
0:45	Router picks the vision slot (visible in the transcript)	“It routed to the vision model, not the coding model — because there's a document attached. That decision is local and deterministic; it doesn't cost a model call.”
1:00	Ingestion: no text layer → rasterise → on-device OCR	“It checked for a text layer first, found none, and fell through to on-device OCR. Nothing is fetched and nothing is uploaded.”
1:30	Findings extracted, with page citations	“Each finding carries the page it came from, so an inspector can go back to the source. An AI answer nobody can check is not usable in a regulated environment.”
2:00	Agent drafts the approval note as a real <code>.docx</code> ; open it	“That's a real Word file, not text in a chat box. That's the artefact the engineer actually has to hand to someone.”
2:30	Run <code>/sovereign report</code>	“And here is the proof. Every outbound attempt this installation has ever made — allowed <i>and</i> refused. All loopback. Zero external.”
2:45	The kicker: point it at a cloud endpoint and watch it refuse	“And if something <i>does</i> try — here's the cloud provider this codebase used to default to. Refused, named, logged, with the subsystem that attempted it. That's what fail-closed means.”

WHY THE KICKER MATTERS MORE THAN THE REST

Anything can look offline when nothing tries to leave. Showing a real attempt being **refused and attributed** is what separates a demo from a proof. Do not cut this even if you are short on time — cut a finding from the report instead.

F.3 Failure plans

If this breaks	Do this
The model is slow or stalls	Keep narrating the architecture from Part E while it runs. Never sit in silence. If it exceeds ~20 s, switch to the recording mid-sentence: “Here's the same run from earlier so we don't burn your time.”
OCR returns nothing	Have a second sample document with a text layer ready. Say what happened: “That one's fighting us — here's a text-layer PDF, which takes the cheap path anyway.”
Anything crashes	One keystroke to the recording. Do not open a log file in front of a judge.
They ask you to try something unscripted	Say yes, once, if it is small. If it fails, that is fine — <i>“that's the edge we haven't hardened yet”</i> is a credible answer. Do not attempt a second one.

Part G — Honest status

READ THIS BEFORE YOU PRESENT ANYTHING

A source-verified audit of our own claims put real readiness at ~57% against an earlier claimed 81%. Six of eight claimed rows were overstated. We corrected the claims rather than the number.

Every team at this hackathon will overstate. A team that states its gaps precisely, and can name the file that would close each one, is the team judges believe about everything else. Feasibility and future progression are both scored criteria — this section is where we win them, not where we lose them.

G.1 Against the problem statement, row by row

PS requirement	State	Evidence / what remains
Model-agnostic backend	BUILT	One OpenAI-compatible seam; capability flags with a conservative floor
New models added without redesign	BUILT	Curated capability table plus environment overrides
Agentic multi-step planning	BUILT	Agent loop, plan manager, sub-agents, failure memory, loop detection
Local tools	BUILT	43 tools
Sandboxed code execution	BUILT	Seatbelt / bubblewrap, network unshared at the autonomous floor
Real deliverables (Word/Excel/PPT/PDF)	BUILT	Four native writers plus a document tool
Runs on-premises, proven	BUILT	Perimeter + ledger + /sovereign , live-verified refusing the cloud provider from two subsystems
Local inference	BUILT	Keyless presets for Ollama, vLLM, LM Studio, llama.cpp
Automatic model selection	PARTIAL	Two-axis routing (slot × weight) ships. A declarative model manifest and a GPU-aware resident-model scheduler with LRU eviction do not — that is the real constraint on one mid-range GPU.
Multimodal: scanned PDF, drawings, handwriting	PARTIAL	PDF rasterisation and OCR ship (Apple Vision / Tesseract). Table extraction and bounding-box provenance are not complete.
Local knowledge base / grounding	GAP	Hybrid retrieval plumbing exists (BM25 + vector + fusion + rerank), but embeddings are remote-first and the corpus is code-shaped, not document-shaped. Air-gapped, semantic search silently becomes keyword search.
Multi-user on-premises server	GAP	Single-user today. Headless mode, RBAC by classification, per-user audit, browser client, GPU-passthrough compose file, offline install bundle — all designed, none built.

G.2 What shipped, and what did not

WP	Scope	State
WP-1	Fail-closed egress at the Node primitives + sovereign sandbox posture	SHIPPED
WP-2	<code>/sovereign</code> command surface, <code>--sovereign</code> flag, boot banner	SHIPPED
WP-3	Local provider presets — Ollama, vLLM, LM Studio, llama.cpp	SHIPPED
WP-4	Task-slot routing above the weight axis	SHIPPED
WP-5	PDF rasterisation, on-device OCR, document reading tool	SHIPPED
WP-6	Local embeddings + organisational document knowledge base	NOT DONE
WP-7	Approval-note template with reference and signature blocks	NOT DONE

G.3 One failing test, and why we did not "fix" it

A retrieval benchmark currently fails. It measures recall on this repository's own source, and adding nine new files shifted one case across the threshold. **It is corpus drift, not a regression.**

We deliberately did not widen the tolerance to make it green. Pinning a threshold to whatever the code currently produces is how a real regression ships next time, unnoticed — the test stops measuring the property and starts recording the number. If a judge sees a red test, this is the answer, and it is a better answer than a green suite would have been.

G.4 Future progression — the roadmap, in order

- Local embeddings and rerank** — a local embedding model through the local runtime, a local cross-encoder reranker, re-embedding migration when the vector space changes, and a **visible degraded-mode banner** so a silent fall back to keyword search becomes impossible.
- Organisational knowledge base** — an incremental, resumable ingester over a watched SMB/NFS mount; per-document classification tags with **access control enforced at retrieval, not at render**; citations that survive into the generated deliverable, so an approval note cites the SOP clause it relied on.
- Registry-driven routing** — a declarative manifest (id, endpoint, modalities, context window, capability flags, VRAM, measured tokens/sec) so adding a model is one entry; a GPU-aware scheduler holding N resident models with LRU eviction and queued admission; a visible routing receipt in the transcript.
- Server deployment** — headless multi-user mode, RBAC by classification level, per-user audit, a browser client over the existing protocol seam, docker-compose with GPU passthrough, and an **offline install bundle** (the current installer fetches from the network, which is the wrong shape for an air-gapped site).
- Domain templates** — approval notes with reference and signature blocks; engineering calculation transcripts with units and shown steps, verified in the sandbox; inspection-report summary formats.

The sequencing is deliberate and worth stating: the egress proof was built **before** local inference, because the ledger is what proves the traffic actually moved on-premises. Building the proof after the claim is how the claim goes unverified.

Part H — Slide-by-slide deck blueprint

14 slides, 10 minutes, with the 3-minute demo sitting at slide 8. Bullets are for the judge's eye; the sentences are yours to say. Never read a slide aloud.

1 Title

On screen: Project name · PS 26117 (MRPL) · team name · six member names with branch · mentor.

Say: Nothing beyond the title. Do not spend pitch time here.

Visual: One strong product screenshot behind the title — the `/sovereign status` output.

2 The problem, as a person experiences it

On screen: One line — *“A refinery's most valuable documents are the ones it is forbidden to upload.”*

Say: Name a real task. An inspection engineer with a scanned, hand-annotated report who needs an approval note by end of day. AI would do this in minutes. They cannot use it, because using it means uploading the document.

Visual: A scanned inspection report with a red “upload” struck through.

3 Why this matters at scale

On screen: Refineries · defence · hospitals · banks · PSUs · legal · government.

Say: This is not one company's inconvenience. The organisations with the most to gain from agentic AI are precisely the ones structurally forbidden from adopting it.

Visual: Sector icons — restrained, not clip-art.

4 Why existing approaches fail

On screen: Four rows: cloud AI · enterprise VPC tiers · local chat UI · local agent frameworks — each with the one-line reason it fails.

Say: Land the last two hard. A local chat window answers questions but cannot act. A local agent framework can act — but nothing stops a single dependency opening a socket, and then the sovereignty claim is false and nobody finds out.

Visual: A 4-row table. No animation.

5 Our solution

On screen: The 90-second pitch from Part C, as four bullets.

Say: An agentic AI workbench that runs entirely on the machine — plans, edits files, runs code in a kernel sandbox, reads scanned documents on-device, produces real Word and Excel deliverables. Driven by open-weight models served locally.

Visual: The system diagram from D.1, simplified to five boxes.

6 What makes it different — the novelty slide

On screen: “Everyone can point a chat window at a local model. We made the absence of network traffic a machine-checked, auditable property.”

Say: The census story from E.2 — we built a guard, then counted its callers, and found two, while sixteen modules opened sockets directly. An opt-in guard is not a perimeter. So we moved it beneath the code.

Visual: The six patched primitives, and below them the ledger line.

7 How it works

On screen: Three layers: in-process perimeter · kernel sandbox for child processes · air-gapped host.

Say: Each layer, one sentence, plus the scope limit we state openly. Then: classification is syntactic and never resolves DNS, because a resolution is itself egress.

Visual: The three-layer table from E.7.

8 LIVE DEMO — 3 minutes

On screen: Nothing. Switch to the terminal.

Say: Part F, verbatim. Unplug the network visibly.

Visual: The product.

9 What we actually built

On screen: 705 modules · ~127k lines · 43 tools · 263 test suites · 13-stage guard pipeline · single binary.

Say: One sentence on scale, then move. Do not linger — this is the least persuasive slide in the deck, and it is here only so the numbers are on record for the Complexity criterion.

Visual: The tool surface grouped by domain.

10 Feasibility

On screen: Runs today on one workstation with Ollama · no runtime dependencies on target · CI on macOS and Linux · SSH-installable.

Say: The delivery format is a decision. An air-gapped server is administered over SSH; a terminal binary installs there and an Electron app does not.

Visual: Deployment topology — laptop / on-prem GPU box, both inside the perimeter.

11 Honest status

On screen: The built / partial / gap table from G.1, trimmed to eight rows.

Say: We audited our own claims and found six of eight overstated. We corrected the claims. Here is exactly what is finished, what is partial, and what is not started.

Visual: Colour-coded status chips. **This slide buys credibility for every other slide.**

12 Impact

On screen: Time saved per report · documents that become searchable · compliance posture · zero data-exit risk.

Say: Frame it as unlocking a use case that is currently at zero, not as a percentage improvement. The baseline is “cannot use AI at all”.

Visual: Before / after of the inspection-report workflow.

13 Future progression

On screen: The five roadmap items from G.4, in order.

Say: Each one names the specific mechanism, not an aspiration. Local embeddings; ACL enforced at retrieval; a model manifest with a GPU-aware scheduler; multi-user server with RBAC; domain templates.

Visual: A simple sequenced roadmap.

14 Ask

On screen: “*Nominate us.*” Plus what we would do with the runway.

Say: Name the two things we would finish before the national round — local embeddings and the organisational knowledge base — and why those two.

Visual: Team photo, contact, repository.

Deck rules

- Maximum 12 words per bullet. If it needs more, it is a sentence you say, not text you show.
- No slide is read aloud. The slide is the judge's channel; your voice is the other one. Running both with the same content wastes one.
- The **problem must land inside 90 seconds**. Slides 2–4 are three minutes of the ten, and they are the most valuable three.
- One idea per slide. If a slide has two ideas, it has one too many.
- Dark terminal screenshots read well projected; light ones wash out. Test on the actual projector if you can.

Part I — Ownership and question bank

I.1 What every member must be able to answer

Judges will pick a member at random. Nobody needs to know every line of code. Everybody needs these seven answers, and they must be consistent across all six of us — **rehearse them together, out loud, or you will contradict each other.**

1. What problem are we solving?

2. What does our product do?

3. Why is it better than existing solutions?

4. What part did I work on?
5. How does the overall system work?

6. What technologies and models are being used?

7. What happens during the demo?

I.2 Suggested ownership split

Fill in the names before you sleep. Every subsystem needs a named owner who answers for it in Q&A.

Role	Owner	Owns in Q&A
Team leader / presenter		Problem framing, impact, the ask, timekeeping. Fields the first question and redirects it to the right owner.
Sovereignty & security		Part E entirely — perimeter, classification, ledger, sandbox posture. The hardest questions land here.
Models & routing		Local providers, the slot × weight router, capability shaping, model choice and its trade-offs
Documents & multimodal		PDF ingestion, OCR backends, citations, the document writers
Agent core & tools		Agent loop, the 43 tools, guard pipeline, memory and the code graph
Demo & presentation		Drives the laptop, owns the recording fallback, owns the deck build

I.3 The question bank

ON SOVEREIGNTY — EXPECT MOST OF THE PRESSURE HERE

Q. How do I know nothing left? Your app could just not be logging it.

A. Three independent checks. One, the ledger records allowed calls as well as refused ones, so it is not a self-serving log. Two, the enforcement is at the Node runtime primitives, so a dependency cannot route around it. Three — and this is the one that does not depend on trusting us — we unplug the network and the task still completes. You can also run a packet capture alongside; the two are complementary, because a capture proves what crossed the wire during those minutes and the ledger proves what the application *tried*, continuously, with the caller attached.

Q. Why not just resolve the hostname and check whether the IP is private?

A. Two reasons. The resolution is itself egress — asking a resolver about a confidential project name leaks that name to whoever runs the resolver. And the answer can change between the check and the connection, which is DNS rebinding. So we classify syntactically, using the WHATWG URL parser so we classify the host the connection will actually be made to, and anything we cannot prove is local is treated as external.

Q. Can't someone just edit the ledger file?

A. Yes. It is tamper-evident at the application level, not tamper-proof, and we say so in the module's own documentation. Making it genuinely append-only is the operating system's job — `chattr +a`, a log shipper, or an audit mount — and it belongs in the deployment guide, not in a claim from us.

Q. What about the shell commands your agent runs? Those are separate processes.

A. Correct, and our in-process perimeter cannot see them. That is why sovereign mode forces the kernel sandbox on with the network denied — `(deny network*)` on macOS, `--unshare-net` on Linux — and refuses the shell tool entirely if no OS sandbox backend is available. We would rather refuse the tool than run it unprotected.

Q. What if the model itself has been trained to exfiltrate data?

A. It has no channel. The model produces text and tool calls; every tool call passes the guard pipeline, and every network path passes the perimeter. A model that *wanted* to send data out would have to get a socket, and there is no socket to get.

ON THE MODELS

Q. Open-weight models are weaker. Is this actually good enough?

A. For this workload, yes — and we designed around their specific failure modes rather than assuming they behave like frontier models. Two concrete examples: the loop recovers tool calls that a weaker model emits as plain JSON in the message body instead of the tool-call channel; and capability shaping strips parameters a given model does not serve before the request is sent, instead of letting the server reject it. Also the alternative is not a better model — the alternative is no AI at all, because the strong models are unusable here.

Q. How does it pick a model? Does that cost you a model call?

A. No — routing is fully local and deterministic, roughly zero milliseconds. Two axes: how much model this turn deserves, and what kind. Slots are tested most-specific first, and vision outranks drafting, because nothing can be drafted from a report the model cannot read. We had an LLM-based classifier earlier; it cost 1.17 seconds per turn and we removed it on that evidence.

Q. What hardware does this need?

A. It runs today on a single workstation with a 7B-class model. On a shared on-premises GPU the real constraint is how many models can stay resident, which is why a GPU-aware resident-model scheduler with LRU eviction is the next routing item on the roadmap — and we have not built it yet.

ON SCOPE AND HONESTY

Q. What doesn't work yet?

A. Answer immediately and specifically — hesitation costs more than the gap does. Local embeddings are not done, so air-gapped the knowledge base degrades to keyword search; the organisational document corpus ingester is not built; it is single-user, not a multi-user server; and there is no approval-note template with signature blocks yet. Then name the file each one lands in.

Q. You have a failing test.

A. Yes — a retrieval benchmark measuring recall on this repository's own source. Nine new files shifted one case across the threshold. It is corpus drift, not a regression, and we deliberately did not widen the tolerance to make it green, because a threshold pinned to whatever the code currently produces stops measuring the property and starts recording the number.

Q. How much of this did you build during the hackathon?

A. Be exact. State what pre-existed and what shipped for this problem statement — the perimeter, the command surface, the local provider presets, the task router and the document reading pipeline. Never imply a 24-hour build of a 127,000-line system; a judge who does the arithmetic will discount everything else you said.

Q. Why should we nominate you over a team with a flashier demo?

A. Because this problem statement's actual difficulty is the proof, not the prototype, and it is the part that determines whether MRPL could ever deploy it. A local chat window is a weekend. An auditable guarantee that a 700-module application with dozens of dependencies made zero external calls is the thing they cannot buy.

ANSWERING RULES

- **Never bluff.** “I don't know, but [owner] does” is a strong answer. An invented one ends the conversation and the judges remember it.
- **Answer the question asked**, in two sentences, then stop. Volunteering more invites a follow-up on ground you did not choose.
- **Redirect deliberately.** The leader takes every question and hands it to the owner. It looks like a team, not six individuals.
- **Concede fast.** If a judge finds a real hole, agree, name it precisely, and say where it lands on the roadmap. Arguing costs more than the hole.

Part J — Glossary

For members whose branch is not CSE, and for anyone explaining this to a judge who knows engineering but not LLMs.

Term	Plain meaning
Agentic	The AI does not just answer — it plans, takes actions with tools, sees the result, and iterates. Answering is a conversation; agentic is work.
Open-weight model	A model whose parameters you can download and run yourself. The opposite of an API you must call over the internet.
On-premises / air-gapped	Running on hardware the organisation owns. Air-gapped means that hardware has no connection to the outside network at all.
Egress	Data leaving the machine. Any outbound network connection.
Fail closed	When the system is unsure, it refuses. The opposite — fail open — allows the action, which is how silent leaks happen.
Loopback	The machine talking to itself (<code>127.0.0.1</code>). The traffic never reaches a network card.
Perimeter	Our interception layer sitting under every possible way the program could open a network connection.
NDJSON	One JSON object per line. Appending is safe and a truncated write costs one line, not the file.
Append-only ledger	A log you only add to. Nothing is edited or deleted, so it can be read as a record of what happened.
Sandbox	An operating-system cage around a running program, limiting what it can read, write and reach. Enforced by the kernel, not by the program's own code.
Seatbelt / bubblewrap	The macOS and Linux sandbox mechanisms respectively.
OCR	Optical Character Recognition — turning a picture of text into text. What makes a scanned report readable by software.
Text layer	Selectable text embedded in a PDF. A scanned PDF has none — it is only an image — which is why OCR is needed.
Multimodal	A model that accepts more than text — here, images of scanned pages and drawings.
Rasterise	Render a PDF page to an image, so a vision model or OCR engine can look at it.
Embedding / vector search	Turning text into numbers so that documents with similar <i>meaning</i> sit close together, letting you find a relevant SOP without matching its exact words.
BM25	Classical keyword ranking. What our search degrades to when embeddings are unavailable — which is the gap in WP-6.
Rerank	A second, more careful pass that reorders search results for relevance.
RAG	Retrieval-Augmented Generation — find the relevant documents first, then let the model answer using them. How grounding and citations work.
AST	Abstract Syntax Tree — a structural representation of code, which is how our editor can address “this function” rather than “these lines”.
Blast radius	What else breaks if you change this. Computed from the code graph.
DNS rebinding	An attack where a hostname resolves to a safe address when checked and a hostile one when connected to.
Prompt injection	Hostile instructions hidden inside content the AI reads, trying to make it act against the user. Our taint guard restricts shell commands once untrusted content has entered the context.
SPOC	Single Point of Contact — the institution's nominated coordinator for SIH.

Team Reference & Presentation Pack — GITAM Hyderabad Internal SIH 2026, 4–5 September 2026.

Prepared against SIH PS 26117 (MRPL): Sovereign On-Premise Agentic AI Workbench using Open-Weight Multimodal LLMs for Confidential Industrial Work.

Every technical claim in Parts C, D, E and G is traceable to shipped source in the project repository. Statuses in Part G reflect a source-verified audit; do not upgrade them in the deck without re-verifying the code.